

Lekcja 06: Inwazja – strzał

Cel

Dokończyć Space Invaders: strzelać, trafiać, wygrywać. Nauczysz się **tablic** – wielu zmiennych pod jedną nazwą.

Co nowego

Masz 24 kosmitów. Każdy może być żywy albo trafiony. 24 zmienne `bool alive1, alive2, ...`? Nie – **tablica**:

```
bool alive[ROWS][COLS];    // 3 x 8 = 24 przegrodek, każda na jedno true/false
alive[0][0] = true;         // rząd 0, kolumna 0
alive[2][7] = false;        // ostatni rząd, ostatnia kolumna (numery zaczynają się od 0!)
if (alive[row][col]) { ... } // odczyt
```

Indeksy liczą się **od zera**: `alive[0]` to pierwszy, `alive[ROWS - 1]` ostatni. `alive[3][0]` przy `ROWS = 3` to błąd, który komputer **nie zawsze zgłosi** – będzie czytać śmieci z pamięci obok. Pętla `for (row = 0; row < ROWS; ...)` pilnuje zakresu sama.

Tablica jednowymiarowa: `int scores[5];` – 5 liczb, `scores[0]` do `scores[4]`. Wypełnianie i czytanie – w pętli `for`. W Scratchu odpowiednikiem tablicy jest **lista**.

Uruchom

Otwórz `gra.cpp`, **Ctrl+Shift+B**. Trzy rzędy kosmitów maszerują skokami. Strzałki ruszają statkiem, **Z** (klawisz A konsoli) strzela żółtym pociskiem. Pocisk przelatuje przez kosmitów – trafienia jeszcze nie ma. To zadanie 2.

Jak to działa

```
bool alive[ROWS][COLS];    // deklaracja - 24 przegrodki

for (int row = 0; row < ROWS; row++)    // setup(): wypełnij wszystkie true
    for (int col = 0; col < COLS; col++)
        alive[row][col] = true;

if (alive[row][col]) {    // frame(): rysuj tylko żywych
    draw_alien(alien_x(col), alien_y(row));
}
```

Pocisk to trzy zmienne: `bullet_active` (czy leci), `bullet_x`, `bullet_y`. Strzał ustawia go na statku, co klatkę leci w górę o 12 px, poza ekranem znika. `pressed(A) && !bullet_active` – strzelasz tylko, gdy w powietrzu nie ma pocisku (`!` to „nie”).

Zadania

1. ★ Zmień kolor pocisku i jego prędkość. Zmień `ROWS` na 4 – czy trzeba coś jeszcze zmieniać? (Nie – to zaleta tablic i pętli.)
2. ★★ **Trafienie.** Wewnątrz `if (bullet_active)` dodaj podwójną pętlę po kosmitach: jeśli `alive[row][col]` i `overlaps(bullet_x - 3, bullet_y, 6, 16, alien_x(col), alien_y(row), ALIEN_W, ALIEN_H)`, to: `alive[row][col] = false; bullet_active = false; score = score + 10;` . (test 1)
3. ★★ Policz żywych: funkcja `int count_alive()` z podwójną pętlą i licznikiem. Gdy zwróci 0 – napis „WYGRALES” i `restart()` po A. Pokaż `watch("alive", count_alive());` .
4. ★★ Kosmici przyspieszają, gdy jest ich mniej: krok bloku co `count_alive() / 2 + 1` klatek zamiast co 10.
5. ★★★ Kosmici strzelają. Tablica `int bombs_y[COLS]` (jedna bomba na kolumnę, `-1` = brak). Co 60 klatek losowa kolumna (`random(0, COLS - 1)`) zrzuci bombę z pozycji najniższego żywego kosmity. Bomba spada o 6 px; gdy trafi statek (`overlaps`) – „PRZEGRALES”. Podpowiedź: bombę rysuj i przesuwaj w pętli `for (int col = 0; col < COLS; col++)` .

Sprawdź sam

Po zadaniu 2 strzał trafia. Trzy strzały bez ruszania statkiem – blok przechodzi nad nim, więc co najmniej jeden trafia:

```
console_sim.exe --game inwazja_strzal --hold A 10 11 --hold A 120 121 --hold A 240 241 --frames 400 --trace 100
```

W ostatniej linii `score` ma być większe od 0. Zadanie **LEKCJA: Sprawdź zadania** robi to samo.

Dla ciekawych

Dlaczego indeksy zaczynają się od 0, a nie od 1? Bo indeks to „ile przegródek od początku”: pierwsza jest 0 od początku. Tak liczy każdy język poza kilkoma bardzo starymi.

Słowniczek

po polsku	w C++
tablica 10 liczb	<code>int t[10];</code>
tablica 3 x 8 wartości prawda/fałsz	<code>bool a[3][8];</code>
pierwszy / ostatni element	<code>t[0] / t[9]</code>
nie (zaprzeczenie)	<code>!warunek</code>
lista w Scratchu	tablica w C++